

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	R.V. Harsha Vardhan	Reg. No.:	192211868
Section / Batch:	CSE - Slot B	Date:	02/09/2026

Code of Conduct

I R.V. Harsha Vardhan (192211868) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: R. Harsha

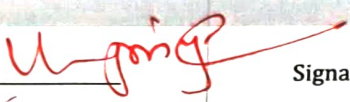
Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:		100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

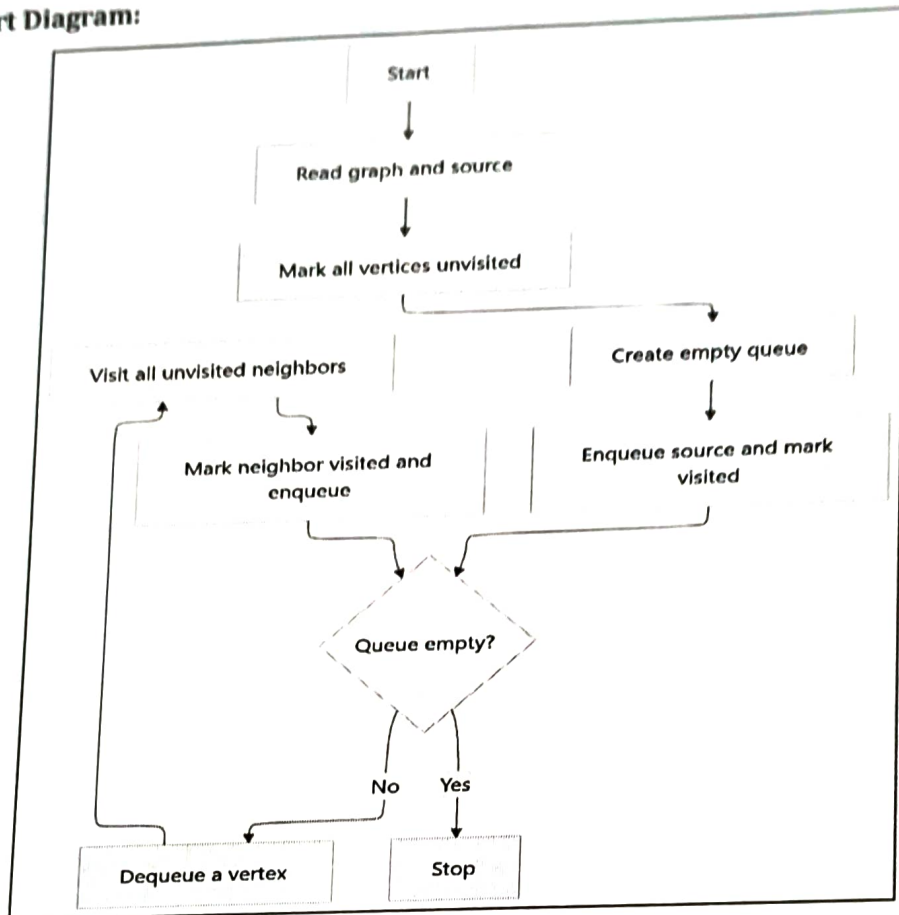
Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A - Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:

Flowchart to code visualization.

① Breadth First Search (BFS) :-

code:

#include <stdio.h>

int main()

{
int n, a[10][10], v[10] = {0}, q[10], r=0, s, i, x;

printf("vertices");

scanf("%d", &n);

printf("Adjacency matrix = \n");

for (i=0; i<n; i++)

for (x=0; x<n; x++)

scanf("%d", &a[i][x]);

printf("source");

scanf("%d", &s);

a[r++] = s; v[s] = 1;

while (F < r) {

x = a[F++];

printf("%d", x);

for (i=0; i<n; i++)

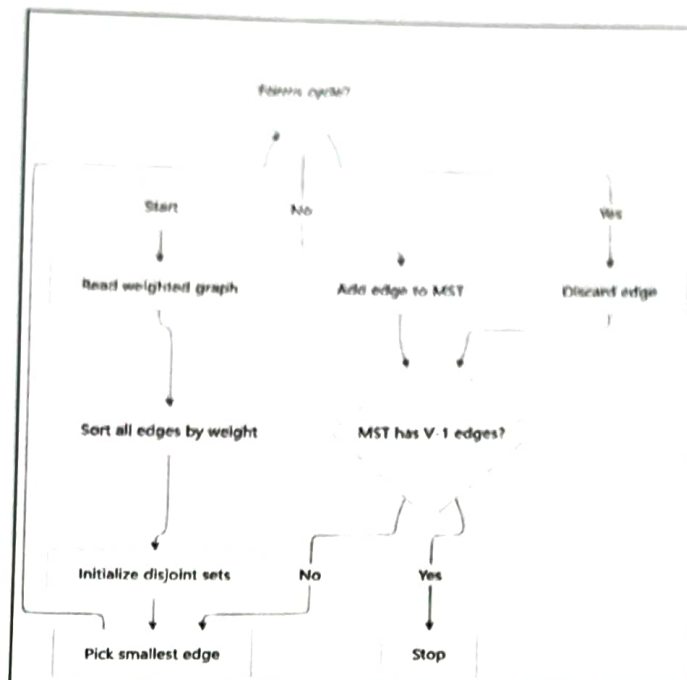
if (a[x][i] && !v[i])

v[i] = 1, a[r++] = i;

}

return 0;

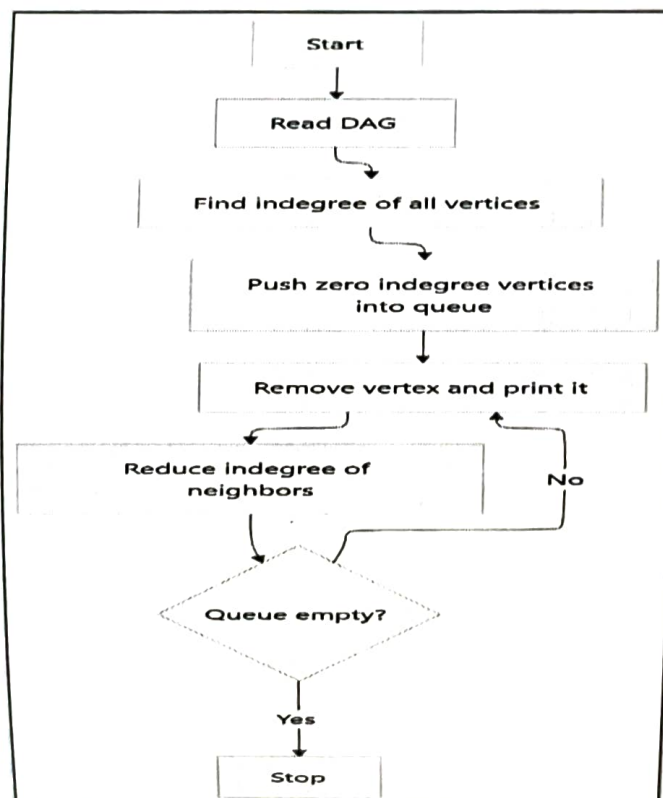
}



3. Convert the flowchart for topological sort into code.

(20 Marks)

Flowchart Diagram:



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

Flowchart Diagram:

Kruskal's Algorithm:

Code:-

```
#include <stdio.h>
```

```
typedef struct {
```

```
    int u, v, w;
```

```
} edge;
```

```
int P[20];
```

```
int find (int x) {
```

```
    return P[x] == x ? x : (P[x] = find(P[x]));
```

```
}
```

```
int main () {
```

```
    edge e[30], t;
```

```
    int n, m, i, j, c = 0;
```

```
    scanf("%d %d", &n, &m);
```

```
    for (i = 0; i < m; i++)
```

```
        scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
```

```
    for (i = 0; i < n; i++) P[i] = i;
```

```
    for (i = 0; i < m - 1; i++)
```

```
        for (j = i + 1; j < m; j++)
```

```
            if (e[i].w > e[j].w)
```

```
                t = e[i], e[i] = e[j], e[j] = t;
```

```
    if (a != b)
```

```
    {
```

```
        P[a] = b;
```

```
        printf("%d %d : %d\n", e[i].u, e[i].v, e[i].w);
```

```
        c++;
```

```
    }
```

```
}
```


Topological Sort

Code:-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, a[10][10]; int[10] = {0}, a[10], x=0, y=0
```

```
scanf("%d", &n);
```

```
for(i=0; i<n; i++)
```

```
for(j=0; j<n; j++) {
```

```
scanf("%d", &a[i][j]);
```

```
if(a[i][j]), in[j]++;
```

```
}
```

```
for(i=0; i<n; i++)
```

```
if(in[i] == 0) a[x++] = i;
```

```
while(x < n)
```

```
printf("%d ", x);
```

```
for(i=0; i<n; i++)
```

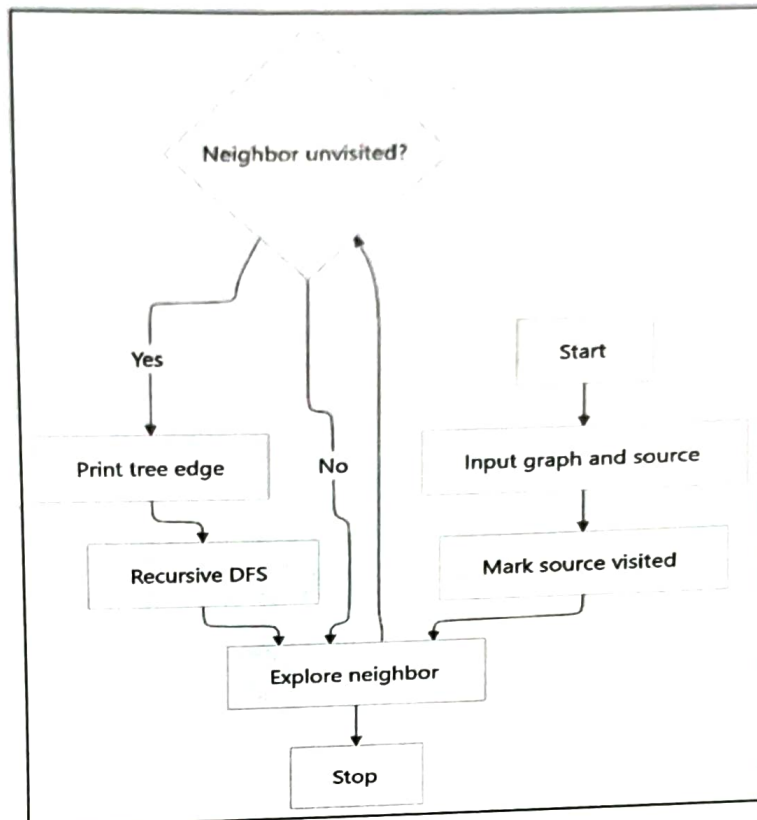
```
if(a[x][i] && --in[i] == 0)
```

```
a[x++] = i;
```

```
}
```

```
return 0;
```

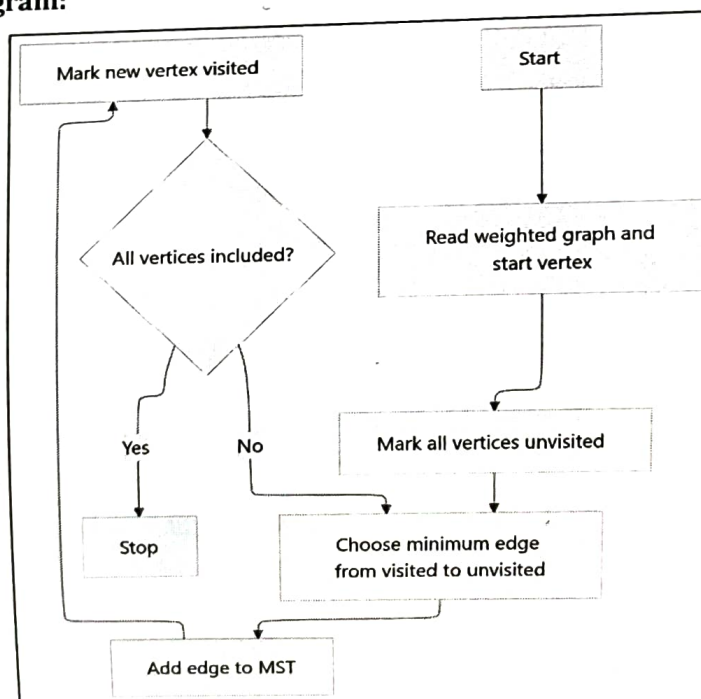
```
}
```



5. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

Flowchart Diagram:



Depth First Search (DFS):

code:

```
#include <stdio.h>
```

```
int a[10][10], v[10], n;
```

```
void dfs (int x) {
```

```
    int i;
```

```
    v[x] = 1;
```

```
    for (i = 0; i < n; i++)
```

```
        if (a[x][i] && !v[i])
```

```
            printf("x d %d \n", x, i);
```

```
            dfs(i);
```

```
}
```

```
}
```

```
int main () {
```

```
    int s, i, j;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    scanf("%d", &s);
```

```
    dfs(s);
```

```
    return 0;
```

```
}
```

prim's algorithm

code:-

#include <stdio.h>

#define INF 9999

int main()

{

int n, a[10][10], p[10], v[10] = {0};

scanf("%d", &n);

for(i=0; i<n; i++)

for(j=0; j<n; j++)

scanf("%d", &a[i][j]);

scanf("%d", &);

v[0] = 1;

for(k=1; k<n; k++){

min = INF

for(i=0; i<n; i++)

if(!v[i],

for(j=0; j<n; j++)

if(!v[j] && a[i][j] & a[i][j] < min)

min = a[i][j] v=1, w=j;

printf("%d - %d : %d\n", u, w, min);

v[w] = 1;

Wgoing

return 0;

}